

Advanced Data Science Notes for Academic Research

A comprehensive reference across 8 advanced topics

Statistical Inference · Python Performance · Feature Engineering
Ensemble ML · Neural Nets · MLOps · Data Pipelines · Visualization

Tailored for: Academia / Research

Contents

01 Statistical Inference & A/B Testing

Hypothesis testing · Power analysis · Effect sizes · Multiple comparisons · Bayesian methods

02 Advanced Python — Pandas & Performance

Vectorization · Memory optimization · Method chaining · Dask & multiprocessing

03 Feature Engineering & Selection

Domain-driven features · Encoding strategies · Leakage prevention · Selection methods

04 Advanced SQL

Window functions · Query optimization · Execution plans · Partitioning

05 Advanced ML — Ensembles & Boosting

Random forests · Gradient boosting · XGBoost/LightGBM · Stacking

06 Advanced ML — Neural Networks

Architectures · Training dynamics · Regularization · Transfer learning

07 MLOps & Model Deployment

Experiment tracking · FastAPI · Docker · MLflow

08 Data Pipelines & Engineering

ETL/ELT · Airflow · dbt · Data quality

09 Advanced Visualization & Dashboarding

Matplotlib mastery · Plotly · D3.js · Publication figures

TOPIC 01

Statistical Inference & A/B Testing

The backbone of credible academic research

1.1 Foundations of Hypothesis Testing

Every inferential claim in research rests on a clear hypothesis framework. The null hypothesis (H_0) represents the status quo; the alternative (H_A) is what you aim to demonstrate.

Key concepts

- **p-value:** Probability of observing data at least as extreme as yours, given H_0 is true. It is NOT the probability that H_0 is true.
- **Type I error (α):** Rejecting H_0 when it is true (false positive). Conventionally set at 0.05, but research contexts may warrant 0.01 or 0.001.
- **Type II error (β):** Failing to reject H_0 when H_A is true (false negative). Power = $1 - \beta$.
- **Test statistic:** A standardized score (t, z, F, χ^2) computed from your data, compared against a reference distribution.

■ Research tip: Always state your α level and justify it before collecting data. Post-hoc α adjustment is a red flag during peer review.

1.2 Choosing the Right Test

Scenario	Recommended Test
Compare 2 group means (independent)	Independent samples t-test or Mann-Whitney U (non-parametric)
Compare 2 group means (paired)	Paired t-test or Wilcoxon signed-rank
Compare 3+ group means	One-way ANOVA → post-hoc (Tukey, Bonferroni)
Compare proportions	Chi-square test or Fisher's exact (small n)
Correlation between continuous variables	Pearson r (normal) or Spearman ρ (ordinal/non-normal)

Regression with continuous outcome	OLS with assumption checks
Binary outcome	Logistic regression or probit

1.3 Power Analysis & Sample Size

Underpowered studies are one of the most common methodological flaws in academic research. Power analysis should be done before data collection.

Power analysis inputs

- **Effect size (d, f, w, r):** The magnitude of the difference you expect. Use pilot data or published literature.
- α : Significance threshold (usually 0.05).
- **Power ($1-\beta$):** Desired probability of detecting the effect. Aim for ≥ 0.80 , ideally 0.90 in publication contexts.
- **Test type:** One-tailed vs two-tailed (almost always two-tailed in academic settings).

Python: power analysis

```
from statsmodels.stats.power import TTestIndPower
analysis = TTestIndPower()
# Solve for sample size
n = analysis.solve_power(
    effect_size=0.5,    # Cohen's d = 0.5 (medium)
    alpha=0.05,
    power=0.80,
    alternative='two-sided'
)
print(f"Required n per group: {n:.0f}") # ~64 per group
# Solve for achieved power given your n
pwr = analysis.solve_power(
    effect_size=0.3,
    alpha=0.05,
    nobs1=40,
    alternative='two-sided'
)
print(f"Achieved power: {pwr:.3f}")
```

1.4 Effect Sizes — Why They Matter More Than p-values

A statistically significant result with a negligible effect size is practically meaningless. Always report both.

Measure	Used For / Interpretation
Cohen's d	Mean differences: $d=0.2$ small, 0.5 medium, 0.8 large
Cohen's f^2	Multiple regression: $f^2=0.02/0.15/0.35$
Cramér's V	Chi-square associations: 0–1 scale
Pearson r / η^2	Correlation and ANOVA proportion of variance explained
Odds Ratio (OR)	Binary outcomes — how much more likely the event is

```
from scipy import stats
import numpy as np
# Cohen's d
def cohens_d(a, b):
    pooled_std = np.sqrt((np.std(a, ddof=1)**2 + np.std(b, ddof=1)**2) / 2)
    return (np.mean(a) - np.mean(b)) / pooled_std
# Eta-squared from one-way ANOVA
f_stat, p_val = stats.f_oneway(group1, group2, group3)
# eta_sq = SS_between / SS_total (requires manual calculation or pingouin)
import pingouin as pg
aov = pg.anova(data=df, dv='score', between='group', detailed=True)
print(aov[['Source', 'F', 'p-unc', 'np2']]) # np2 = partial eta-squared
```

1.5 Multiple Comparisons Correction

When running multiple tests simultaneously, the familywise error rate inflates beyond your nominal α . This is a critical issue in genomics, neuroimaging, and any high-dimensional study.

- **Bonferroni:** Divide α by number of tests. Conservative, controls FWER. Use when tests are independent and few.
- **Holm-Bonferroni:** Sequential version, less conservative than Bonferroni, controls FWER.
- **Benjamini-Hochberg (FDR):** Controls False Discovery Rate — proportion of false positives among rejections. Preferred in high-dimensional settings (genomics, fMRI).

```
from statsmodels.stats.multitest import multipletests
p_values = [0.001, 0.04, 0.06, 0.23, 0.002, 0.08]
# Bonferroni
reject, p_adj, _, _ = multipletests(p_values, method='bonferroni', alpha=0.05)
# Benjamini-Hochberg (FDR)
```

```
reject_fdr, p_adj_fdr, _, _ = multipletests(p_values, method='fdr_bh', alpha=0.05)
for i, (pv, pa, r) in enumerate(zip(p_values, p_adj_fdr, reject_fdr)):
    print(f"Test {i+1}: p={pv:.3f} adj_p={pa:.3f} reject={r}")
```

1.6 A/B Testing in Research Contexts

A/B testing (randomized controlled experiments) is the gold standard for causal claims. The key threat to validity is confounding — eliminated by proper randomization.

Design checklist

- Pre-register your hypothesis, sample size, primary metric, and analysis plan.
- Randomize at the unit of analysis (user, session, patient, lab specimen).
- Check covariate balance post-randomization — run t-tests on baseline characteristics.
- Define stopping rules in advance. Do not peek and stop early based on p-values.
- Account for network effects if units are not independent (SUTVA assumption).

■ **Peeking problem:** Running interim tests without correction inflates Type I error. Use sequential testing (O'Brien-Fleming, Lan-DeMets) or Bayesian stopping rules if you need early stopping.

1.7 Bayesian Inference

Bayesian methods allow you to incorporate prior knowledge and produce probability distributions over parameters — more interpretable than p-values for many research questions.

- **Prior:** Your beliefs about the parameter before seeing data (can be uninformative/flat).
- **Likelihood:** Probability of the data given the parameter values.
- **Posterior:** Updated beliefs after seeing data = Prior × Likelihood (normalized).
- **Credible interval:** 95% CI means 95% probability the true value lies within — unlike frequentist CI.

```
import pymc as pm
import numpy as np

# Bayesian t-test (Kruschke 2013 approach)
y1 = np.array([2.3, 1.9, 2.8, 2.1, 2.5])
y2 = np.array([3.1, 3.4, 2.9, 3.6, 3.2])

with pm.Model() as model:
    mu1 = pm.Normal('mu1', mu=0, sigma=10)
    mu2 = pm.Normal('mu2', mu=0, sigma=10)
    sigma = pm.HalfNormal('sigma', sigma=1)
```

```
obs1 = pm.Normal('obs1', mu=mu1, sigma=sigma, observed=y1)
obs2 = pm.Normal('obs2', mu=mu2, sigma=sigma, observed=y2)

diff = pm.Deterministic('diff', mu1 - mu2)

trace = pm.sample(2000, return_inferencedata=True)
pm.plot_posterior(trace, var_names=['diff'])
```

TOPIC 02

Advanced Python Pandas & Performance

Writing fast, memory-efficient data code for large research datasets

2.1 Vectorization — Eliminate Python Loops

Pure Python loops are 10–100x slower than vectorized NumPy/Pandas operations because they call Python bytecode on each element. Vectorized operations push computation to compiled C/Fortran.

```
import numpy as np
import pandas as pd

# BAD: Python loop – O(n) Python overhead
result = []
for val in df['score']:
    result.append(val ** 2 + 2 * val)

# GOOD: Vectorized – single C call
result = df['score'] ** 2 + 2 * df['score']

# GOOD: np.where for conditional logic
df['grade'] = np.where(df['score'] >= 90, 'A',
                      np.where(df['score'] >= 80, 'B', 'C'))

# GOOD: apply only when truly custom logic is needed
# (still slower than vectorized – avoid if possible)
df['processed'] = df['text'].str.lower().str.strip()
```

2.2 Memory Optimization

Default pandas dtypes are memory-inefficient. A 10M row DataFrame with int64 columns uses 4x the memory of int16 where values fit. This matters when data approaches RAM limits.

```
def optimize_dtypes(df):
    for col in df.select_dtypes('integer').columns:
        df[col] = pd.to_numeric(df[col], downcast='integer')
    for col in df.select_dtypes('float').columns:
        df[col] = pd.to_numeric(df[col], downcast='float')
    for col in df.select_dtypes('object').columns:
        if df[col].nunique() / len(df) < 0.05: # Low cardinality
```



```

        df[col] = df[col].astype('category')
    return df
# Check memory
print(df.memory_usage(deep=True).sum() / 1e6, "MB")
# Read large CSV in chunks
chunks = pd.read_csv('large.csv', chunksize=100_000)
result = pd.concat([process(chunk) for chunk in chunks])

```

2.3 Method Chaining

Method chaining produces readable, pipeline-like data transformations without polluting the namespace with intermediate variables.

```

result = (
    df
    .query("group == 'treatment' and age >= 18")
    .assign(
        log_dose=lambda x: np.log1p(x['dose']),
        duration_days=lambda x: (x['end'] - x['start']).dt.days
    )
    .groupby('participant_id')
    .agg(
        mean_response=('response', 'mean'),
        n_obs=('response', 'count')
    )
    .query("n_obs >= 3")
    .sort_values('mean_response', ascending=False)
    .reset_index()
)

```

2.4 Dask — Larger-than-RAM Datasets

Dask extends the pandas API to datasets that exceed RAM, using lazy evaluation and task graphs. It distributes computation across cores (or a cluster).

```

import dask.dataframe as dd
# Read — lazy, no computation yet
ddf = dd.read_parquet('data/*.parquet')
# Operations mirror pandas
result = (
    ddf
    .groupby('lab_site')

```

```

        .agg({'measurement': ['mean', 'std'], 'n': 'sum'})
    )
# Trigger computation
result_df = result.compute() # Returns a pandas DataFrame
# Persist hot data in memory across operations
ddf = ddf.persist() # Keeps in distributed memory for reuse

```

■ Use Dask when your data is 2–10x RAM. For truly massive data (>100GB), consider Spark or cloud-native solutions like BigQuery.

2.5 Multiprocessing for CPU-bound Tasks

```

from multiprocessing import Pool
import pandas as pd

def process_group(args):
    group_id, group_df = args
    # Expensive per-group computation
    return group_df.assign(zscore=(group_df['val'] - group_df['val'].mean()) / group_df['val'].std())

groups = list(df.groupby('experiment_id'))
with Pool(processes=8) as pool:
    results = pool.map(process_group, groups)
final = pd.concat(results)

# For embarrassingly parallel jobs, joblib is simpler:
from joblib import Parallel, delayed
results = Parallel(n_jobs=-1)(
    delayed(fit_model)(subset) for subset in subsets
)

```

TOPIC 03

Feature Engineering & Selection

Transforming raw data into signals that models can learn from

3.1 Principles of Good Feature Engineering

- **Domain knowledge first:** The best features encode subject-matter understanding, not just statistical patterns.
- **Avoid leakage:** Never include information that would not be available at prediction time.
- **Preserve interpretability:** In academic research, features that reviewers can understand and criticize produce more defensible results.
- **Transformations should be motivated:** Log transforms for skewed data, differencing for non-stationary series, etc.

3.2 Encoding Categorical Variables

Method	When to Use
One-hot encoding	Nominal categories with low cardinality (< 15 levels). Default for linear models.
Ordinal encoding	When category order is meaningful (e.g., education level).
Target encoding	High cardinality nominals. Use with cross-fitting to avoid leakage.
Binary encoding	High cardinality with tree models — reduces dimensionality vs one-hot.
Frequency encoding	Replace category with its frequency. Useful for anonymized IDs.

```
from sklearn.preprocessing import OrdinalEncoder
from category_encoders import TargetEncoder
# Target encoding with cross-validation to prevent leakage
from sklearn.model_selection import KFold
import numpy as np

def cross_val_target_encode(X_train, y_train, col, n_splits=5):
    encoded = np.zeros(len(X_train))
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=42)
    for train_idx, val_idx in kf.split(X_train):
        means = y_train.iloc[train_idx].groupby(X_train[col].iloc[train_idx]).mean()
```

```

        encoded[val_idx] = X_train[col].iloc[val_idx].map(means).fillna(y_train.mean())
    )
    return encoded

```

3.3 Numerical Transformations

```

import numpy as np
from scipy import stats

# Log transform (skewed positive data)
df['log_income'] = np.log1p(df['income']) # log1p handles zeros

# Box-Cox (requires all positive values)
df['bc_col'], lambda_ = stats.boxcox(df['col'] + 1)

# Yeo-Johnson (handles negatives too)
from sklearn.preprocessing import PowerTransformer
pt = PowerTransformer(method='yeo-johnson')
df['transformed'] = pt.fit_transform(df[['col']])

# Binning continuous → ordinal
df['age_group'] = pd.cut(df['age'], bins=[0,18,35,60,100],
                        labels=['youth', 'young_adult', 'middle', 'senior'])

# Interaction terms
df['dose_x_weight'] = df['dose'] * df['body_weight']

# Polynomial features (use sparingly – adds collinearity)
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X[['dose', 'duration']])

```

3.4 Feature Selection Methods

Too many features inflate variance, overfit, and obscure results. Selection is also a form of implicit hypothesis generation in research.

Filter methods — fast, model-agnostic

```

from sklearn.feature_selection import f_classif, mutual_info_classif, SelectKBest

# ANOVA F-test (linear associations)
selector = SelectKBest(f_classif, k=20)
X_selected = selector.fit_transform(X, y)
selected_features = X.columns[selector.get_support()]

# Mutual information (non-linear)
mi_scores = mutual_info_classif(X, y, random_state=42)
feature_importance = pd.Series(mi_scores, index=X.columns).sort_values(ascending=False)

```

SHAP — model-based, interpretable

```
import shap
model.fit(X_train, y_train)
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)
# Mean absolute SHAP as importance
importance = pd.DataFrame({
    'feature': X.columns,
    'importance': np.abs(shap_values).mean(axis=0)
}).sort_values('importance', ascending=False)
shap.summary_plot(shap_values, X_test, plot_type='bar')
```

■ Leakage warning: Never fit any feature engineering (scalers, encoders, imputers) on the full dataset before splitting. Fit on train only, transform train and test.

TOPIC 04

Advanced SQL

Query optimization, window functions, and execution planning

4.1 Window Functions

Window functions perform calculations across a set of rows related to the current row, without collapsing them like GROUP BY. Essential for running totals, rankings, and time-series analysis.

```
-- Running total within each experiment
SELECT
    experiment_id,
    observation_date,
    measurement,
    SUM(measurement) OVER (
        PARTITION BY experiment_id
        ORDER BY observation_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS cumulative_sum,

    -- Lag/lead for differencing
    LAG(measurement, 1) OVER (PARTITION BY experiment_id ORDER BY observation_date)
    AS prev_measurement,
    measurement - LAG(measurement, 1) OVER (PARTITION BY experiment_id ORDER BY observation_date) AS delta,

    -- Rank within group
    RANK() OVER (PARTITION BY experiment_id ORDER BY measurement DESC) AS rank_within_exp,

    -- Moving average (7-day)
    AVG(measurement) OVER (
        PARTITION BY experiment_id
        ORDER BY observation_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS moving_avg_7
FROM measurements;
```

4.2 Common Table Expressions (CTEs)

```
-- Hierarchical CTEs for data transformations
WITH
baseline AS (
    SELECT participant_id, AVG(value) AS baseline_mean
    FROM measurements WHERE phase = 'pre'
    GROUP BY participant_id
),
treatment AS (
    SELECT participant_id, AVG(value) AS treatment_mean
    FROM measurements WHERE phase = 'post'
    GROUP BY participant_id
),
effect_size AS (
    SELECT
        t.participant_id,
        t.treatment_mean - b.baseline_mean AS change,
        (t.treatment_mean - b.baseline_mean) / NULLIF(b.baseline_mean, 0) AS pct_change
    FROM treatment t
    JOIN baseline b USING (participant_id)
)
SELECT * FROM effect_size WHERE ABS(pct_change) > 0.1
ORDER BY pct_change DESC;
```

4.3 Query Optimization

- **Use EXPLAIN / EXPLAIN ANALYZE** to inspect execution plans before optimizing.
- **Index strategically:** Index columns used in WHERE, JOIN ON, and ORDER BY. Avoid indexing low-cardinality columns.
- **Filter early:** Push WHERE clauses as far into subqueries as possible.
- **Avoid SELECT *:** Only select needed columns — reduces I/O and memory.
- **Prefer JOINS over subqueries:** Correlated subqueries execute once per row; joins are optimized by the planner.

```
-- Check execution plan (PostgreSQL)
EXPLAIN ANALYZE
SELECT p.name, COUNT(o.id) as n_outcomes
FROM participants p
JOIN outcomes o ON p.id = o.participant_id
WHERE p.cohort = 2023
GROUP BY p.name;
```

```

-- Create covering index for common query pattern
CREATE INDEX idx_outcomes_participant_date
ON outcomes (participant_id, outcome_date)
INCLUDE (value, status);

-- Partial index for filtered queries
CREATE INDEX idx_active_participants
ON participants (cohort, site)
WHERE status = 'active';

```

4.4 Partitioning for Large Tables

```

-- Range partitioning by year (PostgreSQL)
CREATE TABLE measurements (
    id BIGSERIAL,
    participant_id INT,
    recorded_at TIMESTAMPTZ,
    value FLOAT
) PARTITION BY RANGE (recorded_at);

CREATE TABLE measurements_2022
    PARTITION OF measurements
    FOR VALUES FROM ('2022-01-01') TO ('2023-01-01');

CREATE TABLE measurements_2023
    PARTITION OF measurements
    FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');

-- Queries on date range only scan relevant partitions
SELECT * FROM measurements
WHERE recorded_at BETWEEN '2023-06-01' AND '2023-09-01';

```


TOPIC 05

Advanced ML Ensembles & Boosting

Random forests, gradient boosting, and stacking for research

5.1 Why Ensembles Work — Bias-Variance Decomposition

Prediction error = $\text{Bias}^2 + \text{Variance} + \text{Irreducible noise}$. Ensembles target different sources:

- **Bagging (Random Forests):** Reduces variance by averaging many high-variance, low-bias trees trained on bootstrap samples.
- **Boosting (XGBoost, LightGBM):** Reduces bias by sequentially fitting weak learners on the residuals of prior iterations.
- **Stacking:** Combines multiple diverse models using a meta-learner trained on out-of-fold predictions.

5.2 Random Forests — Key Hyperparameters

Parameter	Effect & Guidance
n_estimators	More trees = lower variance. Diminishing returns after ~300. Start at 500.
max_features	Features per split. $\sqrt{p}$ for classification, $p/3$ for regression.
max_depth	Limits tree depth. None (full trees) is default; tune for regularization.
min_samples_leaf	Minimum samples at leaves. Increase to smooth predictions, reduce overfitting.
oob_score=True	Out-of-bag error — free validation estimate without a held-out set.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

rf = RandomForestClassifier(
    n_estimators=500,
    max_features='sqrt',
    min_samples_leaf=5,
    n_jobs=-1,
    oob_score=True,
    random_state=42
```

```

)
rf.fit(X_train, y_train)
print(f"OOB score: {rf.oob_score_:.4f}")
cv_scores = cross_val_score(rf, X, y, cv=5, scoring='roc_auc')
print(f"CV AUC: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}")

```

5.3 Gradient Boosting — XGBoost & LightGBM

Gradient boosting builds an additive model by fitting each new tree to the negative gradient (pseudo-residuals) of the loss. XGBoost and LightGBM are highly optimized implementations.

```

import xgboost as xgb
import optuna

def objective(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 1000),
        'max_depth': trial.suggest_int('max_depth', 3, 8),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3, log=True),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.6, 1.0),
        'reg_alpha': trial.suggest_float('reg_alpha', 1e-8, 10.0, log=True),
        'reg_lambda': trial.suggest_float('reg_lambda', 1e-8, 10.0, log=True),
        'eval_metric': 'auc', 'use_label_encoder': False
    }
    model = xgb.XGBClassifier(**params, random_state=42)
    score = cross_val_score(model, X_train, y_train, cv=5, scoring='roc_auc').mean()
    return score

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=100)
best_params = study.best_params

```

5.4 Stacking / Model Blending

```

from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
import lightgbm as lgb

estimators = [
    ('rf', RandomForestClassifier(n_estimators=300, n_jobs=-1)),
    ('xgb', xgb.XGBClassifier(n_estimators=200, use_label_encoder=False)),
    ('lgbm', lgb.LGBMClassifier(n_estimators=200)),
]

```

```
# Meta-learner trained on OOF predictions of base models
stacker = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression(C=0.1),
    cv=5,          # 5-fold cross-validation for OOF generation
    stack_method='predict_proba',
    n_jobs=-1
)
stacker.fit(X_train, y_train)
```

TOPIC 06

Advanced ML Neural Networks

Architectures, training dynamics, and when to use them in research

6.1 When to Use Neural Networks in Research

■ For most tabular research data, gradient boosting outperforms neural nets. Use NNs when your data is images, text, audio, or long sequences — or when you have > 100K samples.

6.2 Key Architectures

Architecture	Use Case in Research
MLP (Feedforward)	Tabular data with complex non-linear interactions; survival modeling
CNN	Histology/pathology images, fMRI spatial patterns, spectrogram analysis
LSTM / GRU	EHR sequences, time series, longitudinal clinical data
Transformer	NLP on clinical notes, genomic sequences, proteomics
GNN	Molecular graphs, brain connectivity networks, knowledge graphs
VAE / GAN	Data augmentation for scarce research datasets

6.3 Training Dynamics & Common Pitfalls

- **Learning rate:** Most impactful hyperparameter. Use learning rate finder or cosine annealing schedule.
- **Batch size:** Larger batches = more stable gradients but may generalize worse. 32–256 is typical.
- **Gradient clipping:** Prevents exploding gradients in RNNs. Clip norm to 1.0.
- **Early stopping:** Monitor validation loss; stop when it stops improving for N epochs.

```
import torch
import torch.nn as nn
from torch.optim.lr_scheduler import CosineAnnealingLR
class ResearchMLP(nn.Module):
```

```

def __init__(self, in_features, hidden_dims, out_features, dropout=0.3):
    super().__init__()
    layers = []
    prev = in_features
    for dim in hidden_dims:
        layers += [nn.Linear(prev, dim), nn.BatchNorm1d(dim),
                  nn.ReLU(), nn.Dropout(dropout)]
        prev = dim
    layers.append(nn.Linear(prev, out_features))
    self.net = nn.Sequential(*layers)

    def forward(self, x): return self.net(x)

model = ResearchMLP(in_features=50, hidden_dims=[256, 128, 64], out_features=1)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = CosineAnnealingLR(optimizer, T_max=100)
# Training loop with early stopping
best_val_loss, patience, patience_counter = float('inf'), 15, 0
for epoch in range(500):
    train_loss = train_epoch(model, train_loader, optimizer)
    val_loss = evaluate(model, val_loader)
    scheduler.step()
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        torch.save(model.state_dict(), 'best_model.pt')
        patience_counter = 0
    else:
        patience_counter += 1
        if patience_counter >= patience:
            print(f"Early stop at epoch {epoch}")
            break

```

6.4 Transfer Learning & Fine-tuning

```

from transformers import AutoModelForSequenceClassification, AutoTokenizer
from transformers import Trainer, TrainingArguments

# Fine-tune BioBERT for clinical text classification
model_name = "dmis-lab/biobert-base-cased-v1.2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=3)

# Freeze all but last 2 transformer layers (common strategy)
for name, param in model.named_parameters():
    if 'encoder.layer.10' not in name and 'encoder.layer.11' not in name and 'classifier' not in name:

```

```
        param.requires_grad = False
training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=5,
    per_device_train_batch_size=16,
    warmup_ratio=0.1,
    lr_scheduler_type='cosine',
    evaluation_strategy='epoch',
    load_best_model_at_end=True,
    metric_for_best_model='f1',
)
```

TOPIC 07

MLOps & Model Deployment

Experiment tracking, reproducibility, and serving research models

7.1 Experiment Tracking with MLflow

In research, reproducibility requires logging every hyperparameter, dataset version, and metric. MLflow provides a lightweight tracking server.

```
import mlflow
import mlflow.sklearn
mlflow.set_experiment("my_research_experiment")
with mlflow.start_run(run_name="xgb_v3_tuned"):
    mlflow.log_params({
        'n_estimators': 300,
        'max_depth': 5,
        'learning_rate': 0.05,
        'dataset_version': 'v2.3',
        'feature_set': 'clinical_v4'
    })

    model.fit(X_train, y_train)

    mlflow.log_metrics({
        'train_auc': roc_auc_score(y_train, model.predict_proba(X_train)[:,-1]),
        'val_auc': roc_auc_score(y_val, model.predict_proba(X_val)[:,-1]),
        'val_f1': f1_score(y_val, model.predict(X_val), average='macro')
    })

mlflow.sklearn.log_model(model, "model")
mlflow.log_artifact("feature_importance.png")
```

7.2 Containerization with Docker

Docker ensures your research environment is reproducible — the same code runs identically on your laptop, the HPC cluster, and a collaborator's machine.

```
# Dockerfile for research Python environment
```

```

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
# For an API endpoint
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
# Build and run:
# docker build -t research-model:v1 .
# docker run -p 8000:8000 research-model:v1

```

7.3 Serving Models with FastAPI

```

from fastapi import FastAPI
from pydantic import BaseModel
import joblib
import numpy as np

app = FastAPI(title="Research Model API")
model = joblib.load("model.pkl")
scaler = joblib.load("scaler.pkl")

class PredictionRequest(BaseModel):
    features: list[float]
    participant_id: str | None = None

class PredictionResponse(BaseModel):
    prediction: float
    probability: float
    participant_id: str | None

@app.post("/predict", response_model=PredictionResponse)
async def predict(request: PredictionRequest):
    X = np.array(request.features).reshape(1, -1)
    X_scaled = scaler.transform(X)
    prob = float(model.predict_proba(X_scaled)[0, 1])
    pred = int(prob >= 0.5)
    return PredictionResponse(
        prediction=pred, probability=prob,
        participant_id=request.participant_id
    )

@app.get("/health")
async def health(): return {"status": "ok"}

```


TOPIC 08

Data Pipelines & Engineering

ETL/ELT, Airflow, dbt, and reproducible research data flows

8.1 ETL vs ELT

ETL (Extract-Transform-Load) applies transformations before loading into the target. ELT loads raw data first, then transforms in-place using the warehouse's compute power. ELT is preferred with modern cloud warehouses (BigQuery, Snowflake, Redshift).

8.2 dbt — Transform Data in the Warehouse

dbt (data build tool) allows you to write modular SQL transformations as version-controlled models. Essential for reproducible analytical pipelines.

```
-- models/staging/stg_measurements.sql
{{ config(materialized='view') }}

SELECT
    id,
    participant_id,
    CAST(recorded_at AS TIMESTAMP) AS recorded_at,
    value,
    UPPER(TRIM(measurement_type)) AS measurement_type
FROM {{ source('raw', 'measurements') }}
WHERE value IS NOT NULL
    AND value BETWEEN 0 AND 1000 -- Data quality filter

-- models/marts/analysis_cohort.sql
{{ config(materialized='table') }}

WITH baseline AS (
    SELECT participant_id, AVG(value) AS baseline
    FROM {{ ref('stg_measurements') }}
    WHERE measurement_type = 'BASELINE'
    GROUP BY 1
)

SELECT p.*, b.baseline
FROM {{ ref('stg_participants') }} p
LEFT JOIN baseline b USING (participant_id)
```

```

# Run dbt transformations
# dbt run          -- build all models
# dbt test         -- run data quality tests
# dbt run --select marts.* -- run only mart models
# dbt docs generate && dbt docs serve -- interactive docs

```

8.3 Apache Airflow — Scheduling Research Pipelines

```

from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'research-team',
    'retries': 2,
    'retry_delay': timedelta(minutes=5),
    'email_on_failure': True,
}

with DAG(
    dag_id='weekly_cohort_update',
    default_args=default_args,
    schedule_interval='@weekly',
    start_date=datetime(2024, 1, 1),
    catchup=False,
    tags=['research', 'cohort']
) as dag:

    extract = PythonOperator(
        task_id='extract_from_ehr',
        python_callable=extract_ehr_data
    )

    validate = PythonOperator(
        task_id='validate_data_quality',
        python_callable=run_quality_checks
    )

    transform = PythonOperator(
        task_id='run_dbt_models',
        python_callable=lambda: os.system('dbt run')
    )

    extract >> validate >> transform

```

TOPIC 09

Advanced Visualization & Dashboarding

Publication-quality figures and interactive research dashboards

9.1 Matplotlib Mastery — Publication Figures

Journal figures require specific DPI, font sizes, and color choices that work in both color and grayscale. Configure matplotlib's rcParams once at the top of your script.

```
import matplotlib.pyplot as plt
import matplotlib as mpl
# Publication-quality defaults
mpl.rcParams.update({
    'figure.dpi': 300,
    'font.size': 10,
    'axes.labelsize': 11,
    'axes.titlesize': 12,
    'xtick.labelsize': 9,
    'ytick.labelsize': 9,
    'legend.fontsize': 9,
    'font.family': 'sans-serif',
    'axes.spines.top': False,
    'axes.spines.right': False,
    'axes.grid': True,
    'grid.alpha': 0.3,
    'savefig.bbox': 'tight',
    'savefig.pad_inches': 0.05
})
fig, axes = plt.subplots(1, 2, figsize=(7, 3)) # Width = journal column width
# Always save as vector format (SVG/PDF) + raster
fig.savefig('figure1.pdf', format='pdf')
fig.savefig('figure1.svg', format='svg')
fig.savefig('figure1.png', dpi=300)
```

9.2 Plotly — Interactive Exploratory Figures

```

import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Interactive scatter with facets
fig = px.scatter(
    df, x='baseline', y='outcome', color='treatment_group',
    facet_col='lab_site', trendline='ols',
    hover_data=['participant_id', 'age'],
    title='Baseline vs Outcome by Treatment Group',
    labels={'baseline': 'Baseline Score', 'outcome': 'Outcome Score'},
    color_discrete_map={'control': '#888', 'treatment': '#3C3489'}
)
fig.update_traces(marker=dict(size=6, opacity=0.7))
fig.write_html('interactive_scatter.html')

# Kaplan-Meier survival curve with Plotly
from lifelines import KaplanMeierFitter
fig = go.Figure()
for group, gdf in df.groupby('arm'):
    kmf = KaplanMeierFitter()
    kmf.fit(gdf['time'], gdf['event'], label=group)
    fig.add_scatter(x=kmf.survival_function_.index,
                    y=kmf.survival_function_['KM_estimate'],
                    mode='lines', name=group)

```

9.3 Statistical Visualization Patterns

```

import seaborn as sns

# Raincloud plot (superior to boxplot for research)
import ptitprince as pt
pt.RainCloud(x='group', y='score', data=df, palette='Set2',
             bw=.2, width_viol=.6, ax=ax, orient='h')

# Correlation heatmap with significance masking
import scipy.stats as stats
corr = df.corr()
p_values = df.corr(method=lambda x, y: stats.pearsonr(x, y)[1])
mask = p_values > 0.05 # Mask non-significant correlations
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f',
            cmap='RdBu_r', center=0, vmin=-1, vmax=1,
            linewidths=0.5)

# Forest plot for meta-analysis
effects = [0.34, 0.21, 0.55, 0.18, 0.42]
ci_low  = [0.12, 0.05, 0.30, -0.02, 0.20]

```

```

ci_high = [0.56, 0.37, 0.80, 0.38, 0.64]
studies = ['Study A', 'Study B', 'Study C', 'Study D', 'Study E']
fig, ax = plt.subplots(figsize=(7, 4))
for i, (e, lo, hi, s) in enumerate(zip(effects, ci_low, ci_high, studies)):
    ax.plot([lo, hi], [i, i], 'b-', lw=1.5)
    ax.plot(e, i, 'bs', ms=8)
ax.axvline(0, ls='--', color='gray', lw=0.8)
ax.set_yticks(range(len(studies))); ax.set_yticklabels(studies)
ax.set_xlabel('Effect Size (Cohen's d)')

```

9.4 D3.js for Custom Interactive Figures

D3.js is used when Plotly's built-in chart types are insufficient — custom network graphs, alluvial diagrams, or animated longitudinal visualizations for supplementary web materials.

```

// D3 v7 – basic scatter with brush selection
import * as d3 from "d3";

const svg = d3.select("#chart").append("svg")
    .attr("width", 600).attr("height", 400);

const x = d3.scaleLinear().domain(d3.extent(data, d => d.x)).range([60, 560]);
const y = d3.scaleLinear().domain(d3.extent(data, d => d.y)).range([360, 40]);
svg.append("g").attr("transform", "translate(0,360)").call(d3.axisBottom(x));
svg.append("g").attr("transform", "translate(60,0)").call(d3.axisLeft(y));

svg.selectAll("circle")
    .data(data).join("circle")
    .attr("cx", d => x(d.x)).attr("cy", d => y(d.y))
    .attr("r", 4).attr("fill", d => colorScale(d.group))
    .attr("opacity", 0.75)
    .on("mouseover", (event, d) => tooltip.show(d))
    .on("mouseout", () => tooltip.hide());

```

■ **Key principle:** Every figure should communicate one clear message. Write the caption first, then build the figure to support it.